Secure Exit

SIMATS ENGINEERING

SIMATS Online Programming Lab

JAVA PROGRAMMING

KALAHSTHI RAMYA
192311149RA

CO3-AT1-Assignment

← Back

QUESTIONS

Q1 ✓
Banking Transaction System – Exception Handling
10 marks

Q2 ✓
Hospital Patient Registration – User-Defined Exception
10 marks

Q3 ✓
E-Commerce Order Processing – Built-in Exceptions
10 marks

Q4 ✓
Online Food Delivery – Multiple Threads and Execution Order
10 marks

Q5 ✓
Manufacturing Plant – Producer-Consumer Problem
10 marks

5/5 answered

Q5 Manufacturing Plant – Producer-Consumer Problem — 10 marks

A manufacturing plant has one production thread and one packaging thread. A shared buffer can contain only one product.

Implement the producer-consumer solution using synchronized, wait(), and notify().

The program must:
• Make the producer wait when the buffer is full.
• Make the consumer wait when the buffer is empty.
• Prevent duplicate consumption.
• Ensure that products are consumed only after production.
• Demonstrate correct coordination between the two threads.

Analyze what incorrect behavior could occur if synchronization and wait/notify are removed.
Sample Test Cases:
Input / Scenario Expected Result Analysis Focus
Products = P1, P2, P3 P1 → P2 → P3 are produced and consumed once each Normal execution
Consumer starts first Consumer waits until a product is available Concurrency / design

Buffer capacity = 1; Products = P1, P2 Producer waits while P1 remains unconsumed Exception / boundary

Your Code

```
1 public class Main{
2     public static void main(String[] args){
3         Buffer b = new Buffer();
4 
5         Thread p = new Thread(new Producer(b
6         Thread c = new Thread(new Consumer(b
7 
8         c.start();
9 
10        try{
11            Thread.sleep(100);
12        }catch(InterruptedException e){}
13 
14        p.start();
15    }
16 }
17 
18 class Buffer{
19    private String prod = null;
20 
21    public synchronized void produce(String
22        while(prod != null){
23            try{
24                wait();
25            }catch(InterruptedException e){
26                Thread.currentThread().inter
27            }
28        }
29        prod = newProduct;
30        System.out.println("Produced: "+prod
31 
32        notify();
33    }
34 
35    public synchronized String consume(){
36        while(prod == null){
37            try{
38                wait();
39            }catch(InterruptedException e){
```

```
40                 Thread.currentThread().inter
41             }
42         }
43         String consumedProduct = prod;
44         System.out.println("Consumed: "+cons
45         prod = null;
46         return consumedProduct;
47     }
48 }
49
50 class Producer implements Runnable{
51     private final Buffer b;
52     private final String[] products = {"P1",
53
54     public Producer(Buffer b){
55         this.b = b;
56     }
57     public void run(){
58         for(String p : products){
59             b.produce(p);
60         }
61     }
62 }
63
64 class Consumer implements Runnable{
65     private final Buffer b;
66     private final int count = 3;
67
68     public Consumer(Buffer b){
69         this.b = b;
70     }
71
72     public void run(){
73         for(int i = 0;i<count;i++){
74             b.consume();
75         }
76     }
77 }
```

Input

```
P1
P2
P3
```

Output

Run Save Submit